Dell PowerFlex v3.6.x Gateway

High Availability Technical Notes

March 2023
Rev. 02

DELL Technologies

Notes, cautions, and warnings

NOTE: A NOTE indicates important information that helps you make better use of your product.

CAUTION: A CAUTION indicates either potential damage to hardware or loss of data and tells you how to avoid the problem.

WARNING: A WARNING indicates a potential for property damage, personal injury, or death.

© 2020 – 2023 Dell Inc. or its subsidiaries. All rights reserved. Dell Technologies, Dell, and other trademarks are trademarks of Dell Inc. or its subsidiaries. Other trademarks may be trademarks of their respective owners.

Contents

Chapter 1: Introduction **4**
Overview of the high-availability solution 4

Chapter 2: Example using Apache httpd and Keepalived on Ubuntu **6**
Configuration workflow - Ubuntu environment 6
Update server.xml 6
Create certificates 7
Configure the virtual IP address 8
Enable mod_sll 9
Configure Keepalived 9
Finish the configuration 10

Chapter 3: Example using HAProxy and Keepalived on CentOS **11**
Configure the firewall 11
Prepare the PowerFlex Gateway 12
Configure HAProxy systems 13
Create self-signed certificates for HAProxy 16

1

Introduction

PowerFlex is a software-only solution that uses existing servers' local disks and LAN to create a virtual SAN that has all the benefits of external storage—but at a fraction of the cost and complexity.

PowerFlex utilizes the existing local internal storage and turns it into internal shared block storage. For many workloads, PowerFlex storage is comparable to, or better than, external shared block storage.

The PowerFlex Gateway, which runs on Apache Tomcat, hosts a number of PowerFlex features, including a REST gateway, PowerFlex Installer, and SNMP trap sender. For high availability when two Apache Tomcat instances (PowerFlex Gateway servers) are configured, use Apache httpd.

NOTE: This document discusses procedures needed only in environments in which the REST API commands are used.

Overview of the high-availability solution

To prevent a single point of failure in the Apache httpd, you can use a failover httpd instance that is clustered using one of the many available clustering stacks.

NOTE: When the high-availability solution is used for PowerFlex Gateway, SRS is not supported.

If another gateway is deployed in NonHA mode the SRS functionality will be supported via that IP. Only when the gateway is configured in HA mode for the REST API, as described in this document, with a Virtual IP the SRS functionality will not be supported.

These operating system solutions are described in this document:

- Ubuntu, using Keepalived, a lightweight open source clustering stack for Linux.
- CentOS, using Keepalived and HAProxy

A shared (virtual) IP address is used for both Apache httpd servers. Both of the Apache httpd servers have the same configuration, except that one is defined in the Keepalived configuration as the primary, while the other is defined as the slave. If the primary fails, the secondary becomes the primary, which means the users do not notice any loss of service.

The PowerFlex Gateway servers are configured the same way. According to the configuration in the Apache httpd, one of the PowerFlex Gateway servers (machine2) is the primary and the other (machine1) is the secondary.

In the Ubuntu environment, the mod_jk module in the Apache httpd monitors the PowerFlex Gateway servers and decides to which server to forward the request from the client.

In the CentOS environment, the Keepalived service monitors the PowerFlex Gateway servers and determines to which server to forward the client request.

The following figures illustrate the configuration and topology for the Ubuntu solution:

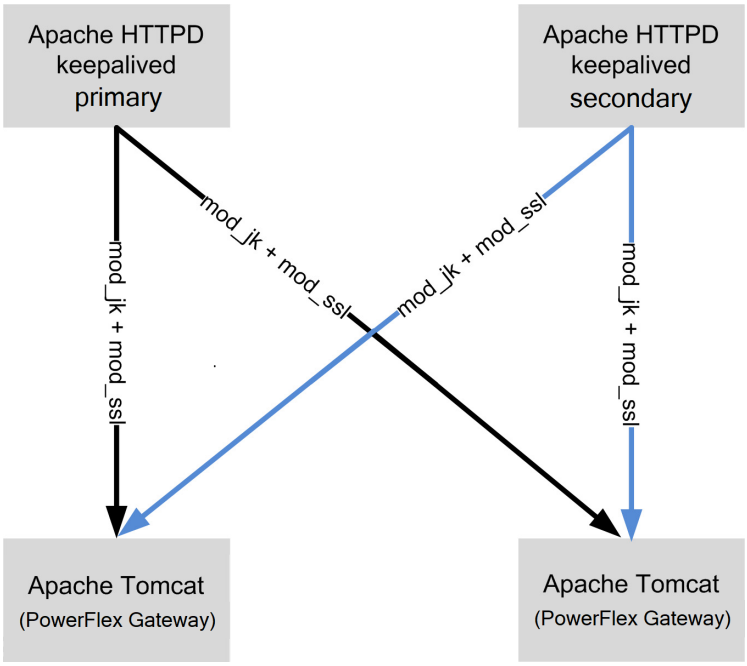


Figure 1. High availability Gateway cluster topology - Ubuntu

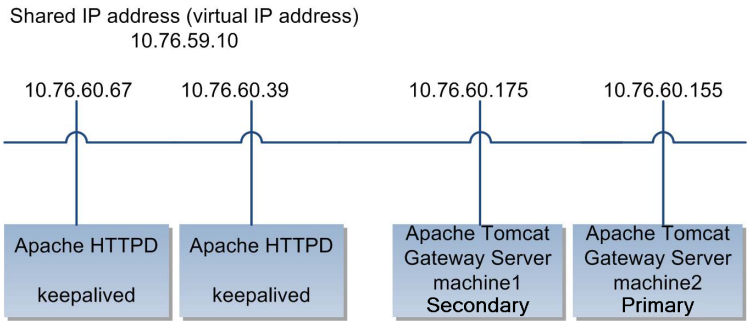


Figure 2. High availability networking example

2

Example using Apache httpd and Keepalived on Ubuntu

The following is a full configuration example for a system running on Ubuntu 14.04 with Apache httpd 2.4.7, with the Keepalived open source tool used for the keep-alive component.

For more information about Keepalived, see http://www.keepalived.org. For more information about Ubuntu and the Apache HTTP Server, see their respective product documentation.

NOTE: The following procedures involve PowerFlex Gateway service restarts.

Configuration workflow - Ubuntu environment

Overview of the configuration workflow to enable high-availability on PowerFlex Gateway servers in the Ubuntu environment.

1. Install Apache httpd.
2. Install mod_jk.
3. Install and enable mod_ssl.
4. Configure the Apache httpd virtual host to use the virtual IP address that is defined for both Apache httpd servers, instead of the default IP addresses.
5. Update the `server.xml` file on the Tomcat servers (PowerFlex Gateway servers) to listen on port 8009 + jvmRoute according to the configuration of the `workers.properties` file used by mod_jk.
6. Install and configure the clustering solution.

Update server.xml

Update the `server.xml` file on the Tomcat servers (PowerFlex Gateway) to listen on port 8009 + jvmRoute according to the configuration of the `workers.properties` file used by the mod_jk module:

Steps

1. Edit the `server.xml` file, located at *<gateway installation directory>*`/conf/server.xml`:
   a. Uncomment and edit the following line:

   ```
   "<!-- <Connector port="8009" protocol="AJP/1.3" redirectPort="443" /> -->"
   ```

   After uncommenting and editing, it should look like this:

   ```
   "<Connector port="8009" protocol="AJP/1.3" redirectPort="${ssl.port}" />"
   ```

   b. Add `jvmRoute="machine1"` to one gateway and `jvmRoute="machine2"` to the other.

   Make sure that the name you used in the `workers.properties` file is the same as the `jvmRoute` value:

   SIO-GW #1:

   ```
   <Engine name="Catalina" defaultHost="localhost" jvmRoute="machine1">
   ```

   SIO-GW #2:

   ```
   <Engine name="Catalina" defaultHost="localhost" jvmRoute="machine2">
   ```

   Note: The `worker.properties` file is configured in the "create certificates" step.

c. If you intend to run Apache httpd on the same machine as the PowerFlex Gateway (not recommended), ensure that the following is commented out to disabled the HTTP connector:

```
<Connector port="${http.port}" protocol="HTTP/1.1"
               connectionTimeout="20000"
               redirectPort="${ssl.port}"
               compression="force"
compressableMimeType="application/json,application/octet-stre am" />
```

NOTE: To prevent port collision with the Apache default SSL port, change the default SSL port to something other than 443 at either Apache or Tomcat.

d. Restart the PowerFlex-gateway service on both PowerFlex Gateway servers

```
service scaleio-gateway restart
```

2. Install Apache httpd:

```
- sudo apt-get install apache2
```

Create certificates

Steps

1. Create a working directory called `certs` at `etc/apache2/certs`.
2. Create `.pem` files:

```
"C:\Program Files\Java\jdk1.7.0_25\bin\keytool.exe" -genkey -alias scaleio -keyalg
RSA -validity 360 -keysize 2048 -dname "CN=<Virtual ip>, OU=ASD, O=EMC, L=Hopkinton,
S=Massachusetts, C=US" -storepass <keystore password> -keypass <keystore password>
-keystore .keystore
```

a. Replace the values for `OU`, `O`, `L`, `S`, and `C` with your credentials.
b. Replace *<keystore password>* with the password you want to use. For example:

```
b2aa330a-e7c7-4e89-86c3-6f2565973cab
```

3. Run the following commands:

NOTE: A password is required.

```
keytool -importkeystore -storepass <keystore password> -keypass <keystore password>
-srckeystore .keystore -destkeystore
intermediate.p12 -deststoretype PKCS12
```

```
openssl pkcs12 -in intermediate.p12 -nocerts -nodes -out filename-key.pem
```

```
openssl pkcs12 -in intermediate.p12 -clcerts -nokeys -out filename-cert.pem
```

4. Copy the `filename-key.pem` and `filename-cert.pem` files you created in the previous step to `/etc/apache2/certs`.
5. Install the mod_jk module:

```
apt-get install libapache2-mod-jk
```

6. Create a `workers.properties` file:

```
touch /etc/apache2/workers.properties
```

7. Add the following content to the `workers.properties` file:

```
worker.list=balance1
worker.machine1.type=ajp13
worker.machine1.host=<machine1_IP>
worker.machine1.port=8009
worker.machine1.lbfactor=1
worker.machine1.activation=disabled
worker.machine2.type=ajp13
worker.machine2.host=<machine2_IP>
worker.machine2.port=8009
worker.machine2.lbfactor=1
worker.machine2.redirect=machine1
worker.balance1.type=lb
worker.balance1.balance_workers=machine1,machine2
```

Where:

- *<machine1_IP>* and *<machine2_IP>* are the IP addresses of the PowerFlex Gateway servers.
- `worker.machine2` is the primary gateway,
- `worker.machine1` is the secondary gateway

8. Update the `/etc/apache2/mods-available/jk.conf` file:
   a. Change `JkWorkersFile` to `JkWorkersFile /etc/apache2/workers.properties`.
   b. Add the following:

```
# Map a worker to a namespace.
JkMount /api/* balance1
JkOptions +ForwardSSLCertChain
```

   c. Delete the following:

```
<Location /jk-status>
        # Inside Location we can omit the URL in JkMount
        JkMount jk-status
        Order deny,allow
        Deny from all
        Allow from 127.0.0.1
</Location>
<Location /jk-manager>
        # Inside Location we can omit the URL in JkMount
        JkMount jk-manager
        Order deny,allow
        Deny from all
        Allow from 127.0.0.1
</Location>
```

Configure the virtual IP address

Configure the virtual IP address for the PowerFlex Gateway servers.

Steps

1. Add the virtual IP address to the `/etc/hosts` file.

   NOTE: Add only the hostname related to the virtual IP.

2. Edit `/etc/apache2/sites-available/default-ssl.conf`:
   a. Find this line in the file:

```
Update <VirtualHost _default_:443>
```

   b. Replace `_default_` with the virtual IP address.

c. Under the `VirtualHost _default` section, add the following lines:

```
SSLCertificateFile /etc/apache2/certs/filename-cert.pem
SSLCertificateKeyFile /etc/apache2/certs/filename-key.pem
JkMountCopy On
JkMount /* balance1
```

Enable mod_sll

Enable mod_sll, that was installed previously.

Steps

Run the following commands:

```
a2enmod ssl
service apache2 restart
a2ensite default default-ssl
service apache2 reload
```

Configure Keepalived

Install Keepalived on every server on which Apache httpd is installed.

Steps

1. Install Keepalived:

```
apt-get install keepalived
```

2. Clear the example in the `keepalived.conf` file:

```
cat /dev/null > /etc/keepalived/keepalived.conf
```

3. Update the `/etc/keepalived/keepalived.conf` file:
   a. Edit `/etc/keepalived/keepalived.conf` on the Apache HTTPS primary server:

```
vrrp_script chk_apache_httpd {
   script "systemctl --no-pager status apache2"
    interval 2
    weight 2
}

vrrp_instance VI_1 {
    interface eth0               # interface to monitor
    state MASTER
    virtual_router_id 51
    priority 101                 # 101 on master, 100 on slave
    virtual_ipaddress {
         <virtual_ip>            # the virtual IP
    }
    track_script {
        chk_apache_httpd
    }
}
```

   b. Edit the `/etc/keepalived/keepalived.conf` file on the Apache HTTPS secondary server:

```
vrrp_script chk_apache_httpd {
   script "systemctl --no-pager status apache2"   # verify the pid is exist or not
```

```
    interval 2                    # check every 2 seconds
    weight 2                      # add 2 points of prio if OK
}

vrrp_instance VI_1 {
    interface eth0                # interface to monitor
    state MASTER
    virtual_router_id 51          # Assign one ID for this route
    priority 101                  # 101 on master, 100 on slave
    virtual_ipaddress {
         <virtual_ip>             # the virtual IP
    }
    track_script {
        chk_apache_httpd
    }
}
```

Finish the configuration

Steps

1. Add this line to the `/etc/sysctl.conf` file:

```
net.ipv4.ip_nonlocal_bind = 1
```

2. Apply the changes:

```
sysctl -p
```

NOTE: This instructs the kernel to bind the non-local IP to the Apache httpd service.

3. Create a symlink to the `/etc` directory:

```
/usr/sbin/keepalived -f /etc/keepalived/keepalived.conf
```

4. Stop, then start, Keepalived:

```
service keepalived stop
service keepalived start
```

3

Example using HAProxy and Keepalived on CentOS

Configure HAProxy and Keepalived on CentOS to provide PowerFlex Gateway high availability.

This section describes how to configure a system running on CentOS, with the Keepalived open source tool used for the keep-alive component.

For more information about Keepalived, see http://www.keepalived.org. For more information about CentOS, see the product documentation.

Configure the firewall

If the firewall is active, you must allow ports. If `firewalld` and `selinux` are disabled, skip this section.

Prerequisites

Consult the firewall vendor documentation on how to open firewall ports.

About this task

These ports must be opened on the firewall: 80, 8080, 443, 28080, 28443, and `vrrp` traffic.

Steps

1. Enable `vrrp` traffic in the firewall:

```
firewall-cmd --permanent --zone=public --add-service=http
firewall-cmd --permanent --zone=public --add-service=https
firewall-cmd --permanent --zone=public --add-port=28443/tcp
firewall-cmd --permanent --zone=public --add-port=28080/tcp
firewall-cmd --permanent --zone=public --add-protocol=vrrp
firewall-cmd --reload
```

2. Validate the configuration:

```
firewall-cmd --list-all
```

Output, similar to the following is displayed:

```
 [root@A59T6290 ~]# firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: bond0 bond0.31 enp2s0f0 enp5s0f0 enp5s0f1
  sources:
  services: dhcpv6-client http https ssh
  ports: 33833/tcp 443/tcp 80/tcp 9011/tcp 6611/tcp 9099/tcp 28080/tcp 28443/tcp
7072/tcp
  protocols: vrrp
  masquerade: no
  forward-ports:
  sourceports:
  icmp-blocks:
  rich rules:
```

3. On both nodes where `selinux` is enabled, run this command:

```
setsebool -P haproxy_connect_any 1
```

4. Verify the new configuration is set to on:

```
/usr/sbin/getsebool -a | grep -i haproxy
```

Output, similar to the following is displayed:

```
[root@A59]# /usr/sbin/getsebool -a | grep -i haproxy
haproxy_connect_any --> on
```

Prepare the PowerFlex Gateway

Prepare the PowerFlex Gateway to enable high availability on CentOS servers.

Prerequisites

Ensure that you have the following:

- Two CentOS servers
- JRE v8 (latest) is installed on the PowerFlex Gateway server.
- Credentials for the PowerFlex system:
  - MDM IP address
  - MDM admin username (default is admin)
  - MDM admin password
  - LIA password

About this task

On each CentOS node, you will install the PowerFlex Gateway and the HA components (HAProxy and Keepalived).

Steps

1. Copy the PowerFlex Gateway RPM to the node.
2. Install the RPM:

```
GATEWAY_ADMIN_PASSWORD=<admin_user_password> rpm -i <rpm_location/rpm_name>
```

3. Add MDM IP addresses to the PowerFlex Gateway:
   a. On the PowerFlex Gateway server, open this file for editing: `/opt/emc/scaleio/gateway/webapps/ROOT/WEB-INF/classes/gatewayUser.properties`
   b. Add all MDM IP addresses to this flag: `mdm.ip.addresses=`
4. Modify the PowerFlex Gateway ports:
   a. On the PowerFlex Gateway server, open this file for editing: `vi /opt/emc/scaleio/gateway/conf/catalina.properties`
   b. Change the PowerFlex Gateway ports:

      From:

      ```
      http.port=80
      ssl.port=443
      ```

      To:

      ```
      http.port=28080
      ssl.port=28443
      ```

   c. Save `catalina.properties` using `wq!`

5. Restart the PowerFlex-gateway service:

```
service scaleio-gateway restart
```

6. Approve the MDM certificates in the PowerFlex Installer:
   a. Log in to the PowerFlex Installer using port 28080 or 28443.
   b. Accept the certificate warning; alternatively, install your own certificate for the Tomcat server.
   c. Enter the default user name, `admin`, and the password defined when the PowerFlex Installer was prepared, then click **Login**.
   d. Click **Maintain**.
   e. Type the MDM and LIA credentials.
   f. Click **Retrieve system topology**.

      A red bar at the top of the screen prompts you to accept certificates.
   g. Click **Approve all**.
   h. Click **Retrieve system topology** again.
   i. Click **Monitor**.

      A red bar appears again at the top of the screen that prompts you to accept certificates.
   j. Click **Approve all**.
   k. Click **Retry Failed**.
   l. Click **Mark Operation Complete**.

Results

The PowerFlex Gateway is installed on the servers, and all security certificates are confirmed.

Configure HAProxy systems

Configure the HAProxy systems, which serve as a load balancer for the PowerFlex Gateway, monitoring the PowerFlex systems and redirecting traffic if a node goes down.

Prerequisites

If you are not using CA certificates, create self-signed certificates, as described in Create self-signed certificates for HAProxy.

About this task

Gateway servers run both HAProxy and Keepalived. Keepalived is responsible for monitoring the status of HAProxy and configuring the virtual IP address as needed.

Steps

1. Install the HAProxy and Keepalived packages:
   a. Run the following, from a shell:

```
$ sudo yum install keepalived haproxy
```

2. Configure HAProxy on the primary node (haproxy1):
   a. Open `/etc/haproxy/haproxy.cfg` for editing.
   b. Modify the value of `gateway1` and `gateway2` to match the PowerFlex Gateway IP addresses.

      NOTE: Dell recommends you first make a copy of the configuration file.

      You can remove the existing `frontend` and `backend` sections, replacing them with sections specific to your needs.

   After editing, the file should look like this:

```
#---------------------------------------------------------------------
# Global settings
#---------------------------------------------------------------------
global
    # to have these messages end up in /var/log/haproxy.log you will
```

```
    # need to:
    #
    # 1) configure syslog to accept network log events.  This is done
    #    by adding the '-r' option to the SYSLOGD_OPTIONS in
    #    /etc/sysconfig/syslog
    #
    # 2) configure local2 events to go to the /var/log/haproxy.log
    #   file. A line like the following can be added to
    #   /etc/sysconfig/syslog
    #
    #    local2.*                       /var/log/haproxy.log
    #
    log         127.0.0.1 local2
    chroot      /var/lib/haproxy
    pidfile     /var/run/haproxy.pid
    maxconn     4000
    user        haproxy
    group       haproxy
    # daemon

    # turn on stats unix socket
    stats socket /var/lib/haproxy/stats
    tune.ssl.default-dh-param 2048
#---------------------------------------------------------------------
# common defaults that all the 'listen' and 'backend' sections will
# use if not designated in their block
#---------------------------------------------------------------------
defaults
    mode                    http
    log                     global
    option                  httplog
    option                  dontlognull
    option                  http-server-close
    option                  redispatch
    retries                 3
    timeout http-request    10s
    timeout queue           1m
    timeout connect         10s
    timeout client          1m
    timeout server          1m
    timeout http-keep-alive 10s
    timeout check           10s
    maxconn                 3000
#---------------------------------------------------------------------
# main frontend which proxys to the backends
#---------------------------------------------------------------------
frontend scaleio
    bind                *:80
    bind *:443 crt-ignore-err all ssl crt /etc/haproxy/certs/haproxy.pem
    option              tcplog
    mode                tcp
    default_backend     siogateway

#---------------------------------------------------------------------
# max connections balancing between the various backends
# this configuration will prefer one system if available,
#    up until it has 256 connections to it
#---------------------------------------------------------------------
backend         siogateway
    mode        tcp
    balance     first
    option      ssl-hello-chk
    server  gateway1 <haproxy1_IP>:28443 maxconn 256 ssl verify none check fall 5
inter 2000 rise 2
    server  gateway2 <haproxy2_IP>:28443 maxconn 256 ssl verify none check fall 5
inter 2000 rise 2
```

3. Copy the `/etc/haproxy/haproxy.cfg` file from the primaryn node (haproxy1) to the secondary node (haproxy2).
4. Set HAProxy to start automatically:

a. On each node, run these commands:

```
$ sudo systemctl enable haproxy
$ sudo systemctl start haproxy
```

5. Configure Keepalived on the primary node (haproxy1):
   a. Create an HAProxy monitor script:
      i. Open `/usr/libexec/keepalived/haproxy-chk.sh` for editing.
      ii. Copy and paste the contents of the script:

```
#!/bin/bash
/usr/sbin/pidof haproxy
exit $?
```

   b. Allow execution of the monitor script:

```
chmod +x /usr/libexec/keepalived/haproxy-chk.sh
```

   c. Open `/etc/keepalived/keepalived.conf` for editing.
   d. Modify these values:

NOTE: Dell recommends you first make a copy of the configuration file.

- `interface`
- `state` (primary on haproxy1)
- `priority` (101 on haproxy1)
- `virtual_ipaddress` (virtual IP address/subnet)

After editing, the file should look like this (comments added):

```
vrrp_script chk_haproxy {
  script "/usr/libexec/keepalived/haproxy-chk.sh"
  interval 2
  weight 2
}

vrrp_instance VI_1 {
  interface ens192 # interface to monitor
  state MASTER # MASTER on haproxy1, BACKUP on haproxy2
  virtual_router_id 51
  priority 101 # 101 on haproxy1, 100 on haproxy2
  virtual_ipaddress {
    <virtual_IP>/24 # virtual ip address and subnet
  }
  track_script {
    chk_haproxy
  }
}
```

6. Configure Keepalived on the secondary node (haproxy2):
   a. Create an HAProxy monitor script:
      i. Open `/usr/libexec/keepalived/haproxy-chk.sh` for editing.
      ii. Copy and paste the contents of the script:

```
#!/bin/bash
/usr/sbin/pidof haproxy
exit $?
```

      iii. Execute the script:

```
chmod +x /usr/libexec/keepalived/haproxy-chk.sh
```

   b. Open `/etc/keepalived/keepalived.conf` for editing.
   c. Modify these values:

NOTE: Dell recommends you first make a copy of the configuration file.

- `interface`
- `state` (BACKUP on haproxy2)
- `priority` (100 on haproxy2)
- `virtual_ipaddress` (virtual IP address/subnet)

After editing, the file should look like this (comments added):

```
vrrp_script chk_haproxy {
  script "/usr/libexec/keepalived/haproxy-chk.sh"
  interval 2
  weight 2
}

vrrp_instance VI_1 {
  interface ens192 # interface to monitor
  state BACKUP # MASTER on haproxy1, BACKUP on haproxy2
  virtual_router_id 51
  priority 100 # 101 on haproxy1, 100 on haproxy2
  virtual_ipaddress {
    <virtual_IP>/24 # virtual ip address and subnet
  }
  track_script {
    chk_haproxy
  }
}
```

7. Modify the Keepalived service to start after networking:
   a. Open `/lib/systemd/system/keepalived.service` for editing.
   b. Change the value of `After=syslog.target` from `network.target` to `network-online.target`
   After editing, the file should look like this:

```
[Unit]
Description=LVS and VRRP High Availability Monitor
After=syslog.target network-online.target

[Service]
Type=forking
PIDFile=/var/run/keepalived.pid
KillMode=process
EnvironmentFile=-/etc/sysconfig/keepalived
ExecStart=/usr/sbin/keepalived $KEEPALIVED_OPTIONS
ExecReload=/bin/kill -HUP $MAINPID

[Install]
WantedBy=multi-user.target
```

8. Set Keepalived to start automatically:
   a. On each node, run these commands:

```
$ sudo systemctl enable keepalived
$ sudo systemctl start keepalived
```

Create self-signed certificates for HAProxy

If not using certificates from a CA, you must generate self-signed certificates for the nodes running HAProxy.

Steps

1. On the first node, create the certificates:
   - If using CentOS 7.x:

```
$ sudo mkdir /etc/haproxy/certs
$ cd /etc/haproxy/certs
$ sudo openssl genrsa -out haproxy.key 1024
$ sudo openssl req -new \
```

```
   -key haproxy.key \
   -out haproxy.csr
$ sudo openssl x509 -req -days 365 \
   -in haproxy.csr \
   -signkey haproxy.key \
   -out haproxy.crt
$ sudo cat haproxy.crt haproxy.key > haproxy.pem
```

- If using CentOS 8.x:

```
$ sudo mkdir /etc/haproxy/certs
$ cd /etc/haproxy/certs
$ sudo openssl genrsa -out haproxy.key 2048
$ sudo openssl req -new \
   -key haproxy.key \
   -out haproxy.csr
$ sudo openssl x509 -req -days 365 \
   -in haproxy.csr \
   -signkey haproxy.key \
   -out haproxy.crt
$ sudo cat haproxy.crt haproxy.key > haproxy.pem
```

2. Copy the entire certificates directory to the other node.